

Nome: João Geiger Piza

Descrição do experimento prático com PyTorch:

O experimento realizado tem como objetivo implementar uma FCN em python e treiná-la em cima do OralEpitheliumDB_Dataset, observando seus resultados.

Descrição do Dataset utilizado:

O OralEpitheliumDB_Dataset é um conjunto de dados público desenvolvido para pesquisas em segmentação e classificação de imagens histopatológicas de displasia epitelial oral. O banco de dados contém 456 imagens obtidas de 30 línguas de camundongos, coradas por hematoxilina e eosina, abrangendo diferentes graus de displasia. Além das imagens originais, o conjunto disponibiliza máscaras de referência com os núcleos celulares anotados manualmente por especialistas e validados por um patologista.

Configuração da FCN utilizada:

A FCN utilizada foi a fcn_resnet50, da biblioteca torchvision.models.segmentation, a função de perda foi a Binary Cross Entropy With Logits, o otimizador usado foi o Adam, a taxa de aprendizado foi de 1e-4, o número de épocas foi de 25 e o tamanho do batch foi 4 (por motivos de performance).

Comparação entre treinamento sem aumento de dados e com aumento geométrico:

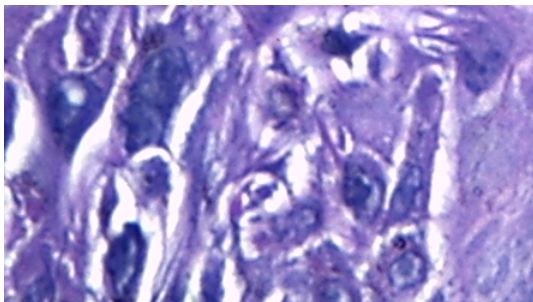
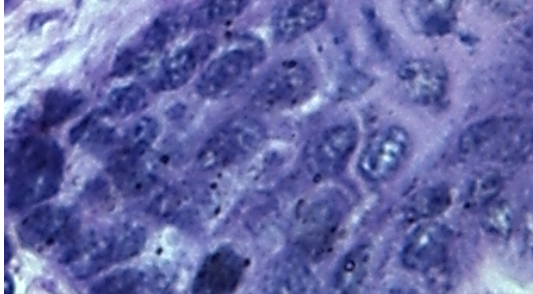
O treinamento com aumento de dados obteve métricas ligeiramente maiores do que o sem aumento de dados, com DICE maior por 0,32%, Acurácia de Pixels maior por 0,38% e IoU maior por 0,48%.

Tabela com os resultados de DICE, acurácia de pixels e IoU:

Experimento	DICE	Acurácia de pixels	IoU	Observações
FCN sem aumento de dados	78,04%	88,08%	64,27%	Nenhuma
FCN com aumento de dados	78,36%	88,46%	64,75%	Aumentos usados: Horizontal flip, Vertical flip e shift scale rotate, todos com probabilidade de 20%.

Exemplos visuais contendo imagem original, máscara real e máscara prevista:

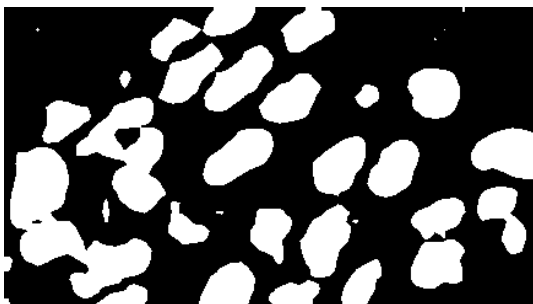
Imagens originais (healthy-02-roi5 e severe-17-roi1, respectivamente):



máscaras reais (healthy-02-roi5 e severe-17-roi1, respectivamente):



máscaras previstas pelo modelo sem aumento de dados (healthy-02-roi5 e severe-17-roi1, respectivamente):





máscaras previstas pelo modelo com aumento de dados (healthy-02-ro15 e severe-17-ro1, respectivamente):



Análise crítica dos resultados:

Os resultados obtidos são satisfatórios para uma FCN, demonstrando DICE por volta de 78%, Acurácia de pixel por volta de 88% e IoU por volta de 64%. Pode-se notar, em algumas máscaras previstas pelo modelo, a presença de núcleos que não existem (núcleos alucinados pelos modelos), o que chama um pouco a atenção.

Motivo do IoU poder ser mais informativo que a acurácia de pixels:

O IoU é muito mais informativo que a acurácia de pixels porque ele avalia o formato e a localização exata do objeto.

Código:

configs.py:

```
import torch
import torch.nn as nn
from torchvision.models.segmentation import fcn_resnet50,
FCN_ResNet50_Weights

SEED = 42

F_LOSS = nn.BCEWithLogitsLoss()
```

```

OTIMIZADOR = torch.optim.Adam
TAXA_APRENDIZADO = 1e-4
NUM_EPOCAS = 25
TAM_BATCH = 4
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
SUAVIZACAO = 1e-8

def pegar_modelo():
    modelo = fcn_resnet50(weights=FCN_ResNet50_Weights.DEFAULT)
    modelo.classifier[4] = nn.Conv2d(512,
                                      1,
                                      kernel_size=(1, 1),
                                      stride=(1, 1)
                                      )
    modelo.aux_classifier[4] = nn.Conv2d(256,
                                          1,
                                          kernel_size=(1, 1),
                                          stride=(1, 1)
                                          )

    return modelo

```

paths.py:

```

import os

DIRETORIO = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(DIRETORIO, "data")
RAW = os.path.join(DATA, "raw")
GOLDEN = os.path.join(DATA, "golden")
MASCARAS = os.path.join(DATA, "mascaras")
MASCARAS_SEM = os.path.join(MASCARAS, "sem_aumento")
MASCARAS_COM = os.path.join(MASCARAS, "com_aumento")

```

prep_dados.py:

```

import os
import multiprocessing

import numpy as np
import albumentations as A
import torch

from torch import nn
from torch.utils.data import Dataset, DataLoader

```

```

from PIL import Image
from albumentations.pytorch import ToTensorV2
from sklearn.model_selection import train_test_split

from paths import RAW, GOLDEN
from configs import SEED, TAM_BATCH

class ConjuntoDadosTif(Dataset):
    def __init__(self, lista_arquivos, transformacoes):
        self.lista_arquivos = lista_arquivos
        self.transformacoes = transformacoes

    def __len__(self):
        return len(self.lista_arquivos)

    def __getitem__(self, indice):
        nome = self.lista_arquivos[indice]

        path_completo_imagem = os.path.join(RAW, f"{nome}.tif")
        path_completo_mascara = os.path.join(GOLDEN, f"{nome}.png")

        imagem =
np.array(Image.open(path_completo_imagem).convert("RGB"))
        mascara =
np.array(Image.open(path_completo_mascara).convert("L"))

        mascara_binaria = (mascara > 0).astype(np.int64)

        aumentado = self.transformacoes(image=imagem,
mask=mascara_binaria)
        imagem_transformada = aumentado['image']
        mascara_transformada = aumentado['mask'].long()

        return imagem_transformada, mascara_transformada

def pegar_loades(aumento: bool):
    transformacoes_sem_aumento = A.Compose([
        A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),
        ToTensorV2()
    ])

```

```

transformacoes_com_aumento = A.Compose([
    A.HorizontalFlip(p=0.2),
    A.VerticalFlip(p=0.2),
    A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.1,
rotate_limit=45, p=0.2),
    A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,
0.225)),
    ToTensorV2()
])

arquivos_totais = [
    os.path.splitext(arquivo)[0]
    for arquivo in os.listdir(RAW)
    if arquivo.endswith('.tif')
]

arquivos_treino, arquivos_teste = train_test_split(
    arquivos_totais,
    test_size=0.2,
    random_state=SEED
)

if not aumento:
    dataset_treino_normal = ConjuntoDadosTif(arquivos_treino,
transformacoes_sem_aumento)
    dataset_teste_normal = ConjuntoDadosTif(arquivos_teste,
transformacoes_sem_aumento)
    carregador_treino = DataLoader(dataset_treino_normal,
batch_size=TAM_BATCH, shuffle=True, pin_memory=True)
    carregador_teste = DataLoader(dataset_teste_normal,
batch_size=TAM_BATCH, shuffle=False, pin_memory=True)
else:
    dataset_treino_aumentado = ConjuntoDadosTif(arquivos_treino,
transformacoes_com_aumento)
    dataset_teste_aumentado = ConjuntoDadosTif(arquivos_teste,
transformacoes_sem_aumento)
    carregador_treino = DataLoader(dataset_treino_aumentado,
batch_size=TAM_BATCH, shuffle=True, pin_memory=True)
    carregador_teste = DataLoader(dataset_teste_aumentado,
batch_size=TAM_BATCH, shuffle=False, pin_memory=True)

return carregador_treino, carregador_teste

```

metricas.py:

```
import torch

from configs import SUAVIZACAO

def calcular_dice(saidas, mascaras):
    previsoos = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

    intersecao = (previsoos * mascaras).sum()
    soma = previsoos.sum() + mascaras.sum()

    dice = (2.0 * intersecao + SUAVIZACAO) / (soma + SUAVIZACAO)
    return dice.item()

def calcular_acuracia_pixel(saidas, mascaras):
    previsoos = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

    corretos = (previsoos == mascaras).sum()
    total = mascaras.numel()

    acuracia = corretos / total
    return acuracia.item()

def calcular_iou(saidas, mascaras):
    previsoos = (saidas > 0).float().flatten()
    mascaras = mascaras.flatten()

    intersecao = (previsoos * mascaras).sum()
    uniao = previsoos.sum() + mascaras.sum() - intersecao

    iou = (intersecao + SUAVIZACAO) / (uniao + SUAVIZACAO)
    return iou.item()
```

treino.py:

```
import torch

import configs as cfg
from prep_dados import pegar_loades
from metricas import calcular_dice, calcular_acuracia_pixel,
```

```

calcular_iou

def treinar(aumento: bool):
    device = cfg.DEVICE
    print(f"Treinando com: {device}")
    modelo = cfg.pegar_modelo()
    modelo.to(device)
    otim = cfg.OTIMIZADOR(modelo.parameters(), lr=cfg.TAXA_APRENDIZADO)
    loader_treino, loader_teste = pegar_loades(aumento)

    for epoca in range(cfg.NUM_EPOCAS):
        modelo.train()
        perda_total_treino = 0.0

        for img, msc in loader_treino:
            img = img.to(device)
            msc = msc.unsqueeze(1).to(device).float()

            otim.zero_grad()

            saidas = modelo(img)
            saida_p = saidas['out']
            saida_a = saidas['aux']

            perda_p = cfg.F_LOSS(saida_p, msc)
            perda_a = cfg.F_LOSS(saida_a, msc)
            perda = perda_p + 0.4*perda_a

            perda.backward()
            otim.step()
            perda_total_treino += perda.item()

        modelo.eval()
        perda_total_teste = 0.0
        dice_total = 0.0
        acuracia_total = 0.0
        iou_total = 0.0

        with torch.no_grad():
            for img, msc in loader_teste:
                img = img.to(device)

```

```

        msc = msc.unsqueeze(1).to(device).float()

        saidas = modelo(img)
        saida_p =saidas['out']

        perda = cfg.F_LOSS(saida_p, msc)
        perda_total_teste += perda.item()

        dice_total += calcular_dice(saida_p, msc)
        acuracia_total += calcular_acuracia_pixel(saida_p, msc)
        iou_total += calcular_iou(saida_p, msc)

    print(f'Epoca {epoca+1}:')
    print(f'- Perda no treino:
{perda_total_treino/len(loader_treino):.4f}')
    print(f'- Perda no teste:
{perda_total_teste/len(loader_teste):.4f}')
    print(f'- DICE: {dice_total/len(loader_teste):.4f}')
    print(f'- Pixel Accuracy:
{acuracia_total/len(loader_teste):.4f}')
    print(f'- IoU: {iou_total/len(loader_teste):.4f}')

return modelo

```

main.py:

```

import os
import torch
import numpy as np
from PIL import Image
import albumentations as A
from albumentations.pytorch import ToTensorV2

import configs as cfg
from paths import RAW, MASCARAS_SEM, MASCARAS_COM
from treino import treinar

def gerar_mascaras(modelo, pasta_destino):
    os.makedirs(pasta_destino, exist_ok=True)

    device = cfg.DEVICE
    modelo.eval()

    transformacao_inferencia = A.Compose([
        A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224,

```

```

0.225)),
    ToTensorV2()
])

arquivos = [arquivo for arquivo in os.listdir(RAW) if
arquivo.endswith('.tif')]

with torch.no_grad():
    for arquivo in arquivos:
        caminho_imagem = os.path.join(RAW, arquivo)

        imagem_pil = Image.open(caminho_imagem).convert("RGB")
        imagem_np = np.array(imagem_pil)

        tensor_imagem =
transformacao_inferencia(image=imagem_np)['image']
        tensor_imagem = tensor_imagem.unsqueeze(0).to(device)

        saidas = modelo(tensor_imagem)
        saida_p = saidas['out']

        mascara_predita = (saida_p > 0).squeeze().cpu().numpy()

        mascara_visivel = (mascara_predita * 255).astype(np.uint8)

        nome_puro = os.path.splitext(arquivo)[0]
        caminho_salvar = os.path.join(pasta_destino,
f"{nome_puro}.png")
        Image.fromarray(mascara_visivel).save(caminho_salvar)

if __name__ == '__main__':
    print("Iniciando treinamento SEM aumento de dados...")
    modelo_sem = treinar(False)

    print("\nIniciando treinamento COM aumento de dados...")
    modelo_com = treinar(True)

    print("\nGerando previsões para o modelo SEM aumento...")
    gerar_mascaras(modelo_sem, MASCARAS_SEM)

    print("Gerando previsões para o modelo COM aumento...")
    gerar_mascaras(modelo_com, MASCARAS_COM)

```

```
print("Processo finalizado com sucesso!")
```